Al-Tarqi SkyCast Weather Web Application

Software Construction Project Report

Sohar University

College of Computing and Information Technology

Course: Software Construction

Course Code: COMP3131

Student Name: [Student Name]

Student ID: [Student ID]

Instructor: [Instructor Name]

Academic Semester: [Semester]

Submission Date: [Submission Date]

Prepared from the implemented React, TypeScript, Node.js and Express project in the shared workspace.

Abstract

This report documents Al-Tarqi SkyCast, a responsive weather web application presented through the SkyCast interface. The system addresses the practical need for a concise, location-aware weather overview and a simple path from city search to an outdoor decision. The implemented application opens without authentication, obtains current and forecast data from Open-Meteo, supports Arabic and English place search, resolves browser coordinates through reverse geocoding, displays current and hourly conditions, stores favourite places in browser localStorage, renders an interactive Leaflet map, and provides a weather assistant through a server-side Mibyan AI adapter. The server protects provider credentials and exposes application-specific routes to the React client.

The project uses React, TypeScript, Vite, Node.js, Express, Leaflet, OpenStreetMap tiles, Lucide icons and CSS. The assistant uses Mibyan's OpenAI-compatible Chat Completions contract with a weather-aware prompt and a deterministic fallback when the provider is unavailable or the project key is unauthorised. This distinction is important: the integration is implemented, but the current workspace key was rejected during live verification, so the report does not claim successful model output.

The report separates verified implementation facts from future proposals. It covers requirements, architecture, UML-style diagrams, customer journey, UX, implementation, testing, quality, security, limitations and future work. It demonstrates Software Construction through clear API boundaries, environment configuration, reusable UI functions, error handling and iterative verification.

Table of Contents

If page numbers do not refresh automatically, open the document in Word and update the table field.

1 Introduction

1.1 Background

Al-Tarqi SkyCast is a browser-based weather dashboard whose visible product identity is SkyCast Weather. It was constructed as an interactive utility rather than a static HTML mock-up. The visitor enters directly, chooses a place, reads the current and near-term weather, saves useful places and asks the assistant for an interpretation.

1.2 Problem Statement

Weather information is often fragmented between a current-temperature screen, a forecast view, a search form and a map. The project addresses this fragmentation by placing the most relevant information in one visual dashboard. The problem is framed as an engineering requirement: a visitor needs a fast, readable and location-aware way to interpret current weather and plan an activity.

1.3 Aim and Objectives

The aim is to construct a responsive weather tool that gives visitors a reliable current view, supports place exploration and explains the available data in natural language when the AI provider is available.

- Open directly without registration.
- Search places in Arabic and English with country and region disambiguation.
- Use browser location only after an explicit user action.
- Display current temperature, apparent temperature, humidity, high/low, sunrise/sunset and hourly data.
- Persist favourite places and theme preference locally.
- Display an interactive map and selected-place marker.
- Connect the weather assistant through a server-side Mibyan adapter.
- Keep the hourly strip scrollable inside the page on phone widths.

1.4 Scope and Report Structure

The implemented scope is a single-page dashboard and Express API. It excludes accounts, database records, payments, push notifications, administrative screens and production deployment. Later sections document the actual code and distinguish future work from delivered behaviour.

2 Problem Analysis

The primary user is an anonymous visitor who wants an immediate answer. The interface therefore privileges a current card, a compact hourly timeline and activity suggestions. Search results include region and country because city names are not necessarily unique. Favourites are deliberately device-local because no identity or database layer exists.

Need	Implemented response	Boundary
Immediate orientation	Current card and selected city	No personal profile
Place discovery	Arabic/English geocoding	Provider quality varies
Comparison	Saved rows with temperatures	No advanced analytics
Interpretation	Activity cards and chat	Informational only
Spatial context	Leaflet map marker	No weather raster layers

3 Objectives and Scope

ID	Objective	Success criterion	Status
OBJ-01	Fast access	Root dashboard opens without login	Implemented
OBJ-02	Current weather	Values populated from /api/weather	Implemented
OBJ-03	Global search	Arabic/English results include region	Implemented
OBJ-04	Saved places	localStorage survives refresh	Implemented
OBJ-05	Map context	Marker updates after selection	Implemented
OBJ-06	Assistant	Valid Mibyan key returns completion	Partially verified
OBJ-07	Accounts and alerts	Future backend scope	Future

“Partially verified” means the adapter and request path exist, but live Mibyan output was not confirmed because the current project key was rejected. This is not reported as a successful AI result.

4 Stakeholder Analysis

Stakeholder	Needs	Impact
Website visitor	Fast weather, search, readable UI, no account	High
Student developer	Maintainable code and provider configuration	High
Course instructor	Evidence of construction, testing and limitations	Medium
External providers	Valid requests and credential protection	Medium
Future maintainer	Documented endpoints and modular boundaries	Medium

No administrator, merchant or service-provider workflow was found in the inspected source, so these roles are not claimed as implemented stakeholders.

5 Requirements Analysis

5.1 Functional Requirements

ID	Requirement	Actor	Expected outcome	Status	Verification
FR-01	Display selected-place weather summary	Visitor	Current card and metrics render	Implemented	UI/API test
FR-02	Search a place in Arabic or English	Visitor	Disambiguated results appear	Implemented	Integration test
FR-03	Select result and reload weather	Visitor	Cards and map update	Implemented	System test
FR-04	Request device location	Visitor	Reverse-geocoded area is shown	Implemented; permission-dependent	Browser test
FR-05	Browse 24-hour	Visitor	Page does not scroll	Implemented	Responsive test

	strip internally		horizontally		
FR-06	Add/remove saved places	Visitor	localStorage list updates	Implemented	Functional test
FR-07	Toggle units and theme	Visitor	Visible state changes	Partially implemented	UI test
FR-08	Show map marker	Visitor	Marker follows place	Implemented	Integration test
FR-09	Ask assistant question	Visitor	Model or fallback answer appears	Partially verified	API test
FR-10	Handle provider failure safely	System	Fallback without key exposure	Implemented	Failure test

5.2 Non-Functional Requirements

Attribute	Interpretation	Evidence/status
Usability	Understand dashboard without registration	Mobile/browser review; verified for tested view
Performance	Keep client work and provider routes concise	No benchmark; not fully tested
Reliability	Maintain useful chat when provider fails	Fallback observed
Maintainability	Separate client, server, types and styles	Verified by source inspection
Compatibility	Work at desktop and phone widths	Phone viewport checked
Security	Keep keys server-side	Mibyan key read by Express only
Accessibility	Readable contrast and labelled controls	Partial; full audit not executed

5.3 Traceability Matrix

Requirement	Component	Test case	Status
FR-01	App.tsx current card	TC-01	Verified
FR-02	/api/search + ask bar	TC-02	Verified
FR-04	useLocation + reverse-geocode	TC-03	Permission-dependent
FR-05	hour-columns CSS	TC-04	Verified
FR-06	favorites/localStorage	TC-05	Verified
FR-08	WeatherMap	TC-06	Implemented
FR-09	/api/chat + Mibyan	TC-07	External key failed
FR-10	Server fallback	TC-08	Verified

6 Development Methodology

No formal Scrum board, sprint record or signed lifecycle document was found. It would therefore be inaccurate to claim a formal Agile, Scrum or Waterfall process. The available evidence shows an iterative construction loop: visual feedback was converted into requirements, bounded code changes were made, builds and browser checks were executed, and external-provider failures were diagnosed.

Figure 8 Iterative Development Cycle

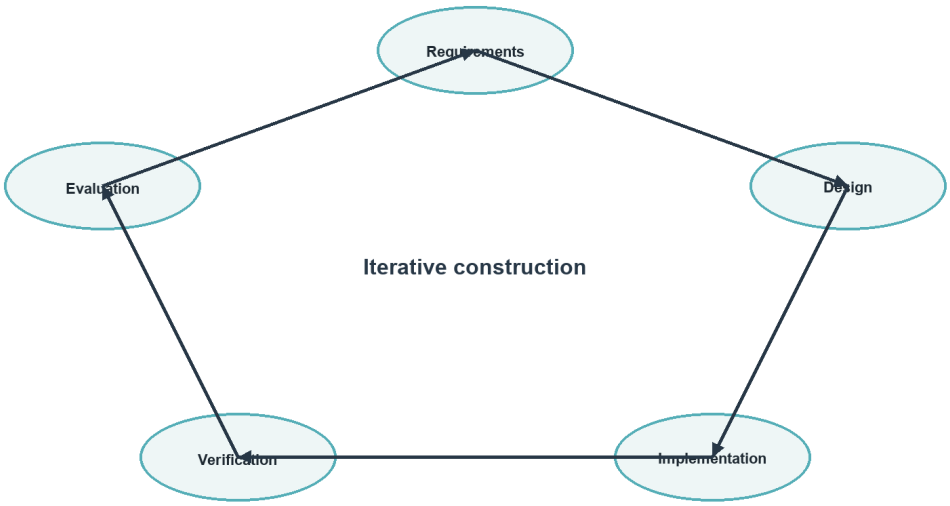


Figure 8: Iterative construction cycle inferred from the project work

7 System Architecture

The browser contains the React presentation layer and calls local Express routes. Express calls Open-Meteo for weather and geocoding, Nominatim for reverse geocoding, and Mibyan for chat. Leaflet loads OpenStreetMap tiles from the browser. Provider keys remain on the server.

Figure 1 System Architecture Diagram

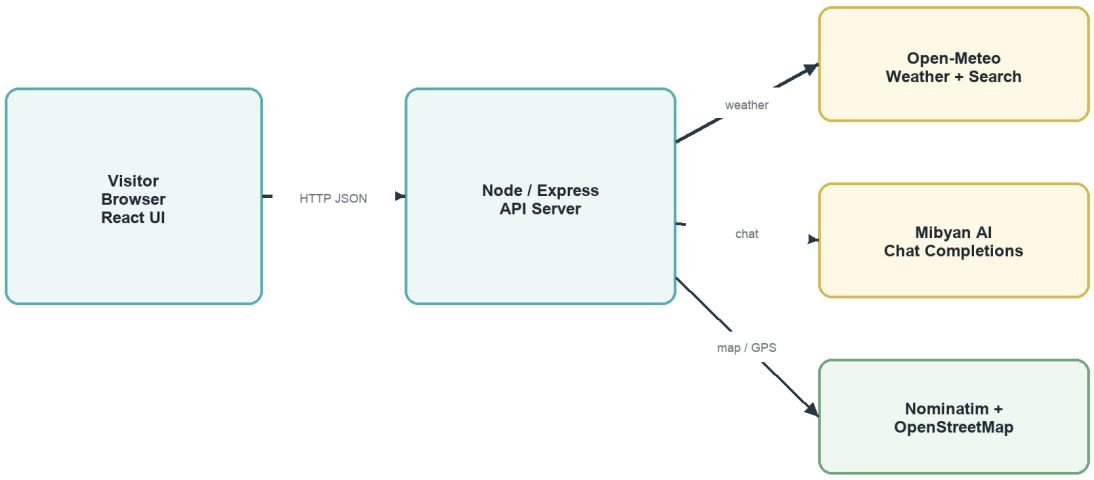


Figure 1: System architecture and external service boundaries

Component	Responsibility	Evidence
React/Vite client	UI, state, interactions	src/App.tsx and styles.css

Express server	Provider boundary and validation	server/index.ts
localStorage	Anonymous place/theme persistence	skycast:* keys
Open-Meteo	Forecast and geocoding	/api/weather and /api/search
Nominatim	Coordinate-to-area name	/api/reverse-geocode
Mibyan AI	Weather chat completion	/api/chat and MIBYAN_*
Leaflet/OpenStreetMap	Map and tiles	WeatherMap

8 Use Case Analysis

The primary actor is Visitor. The visitor is anonymous; there is no implemented account or administrator actor.

Figure 2 Use Case Diagram

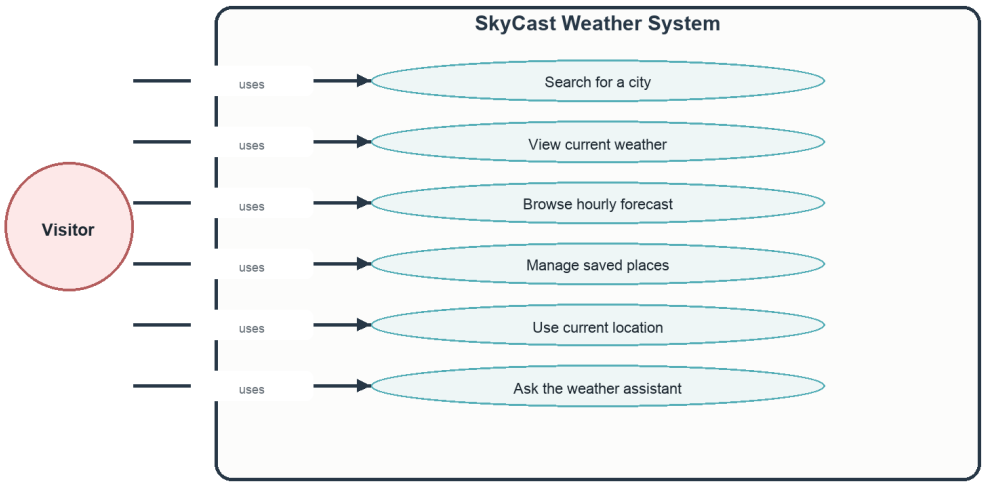


Figure 2: Use case diagram for the Al-Tarqi SkyCast website

UC-01 Search for a place

The visitor enters Arabic or English text; the client debounces it; Express queries both language variants; results are deduplicated by coordinates; the visitor selects a result. Empty results and provider failure are handled without crashing the page.

UC-02 View current weather

After a place is selected, the client loads current, hourly and daily payloads and renders the cards. Provider failure resolves loading without inventing values.

UC-03 Use current location

The visitor presses the location action, grants permission, and receives reverse-geocoded coordinates. Permission denial or reverse-geocoding failure produces a safe generic label.

UC-04 Ask the assistant

The visitor submits a question. The server sends location and weather context to Mibyan. If the provider key is missing or rejected, a bounded weather-data fallback is returned.

UC-05 Manage saved places

The visitor adds a search result or removes a saved row. React state and localStorage are updated, with duplicate place IDs ignored.

9 Customer Journey and User Flow

No field research or analytics dataset was supplied. The following journey is therefore proposed and illustrative, not a measured customer study.

Figure 3 Proposed Customer Journey Map

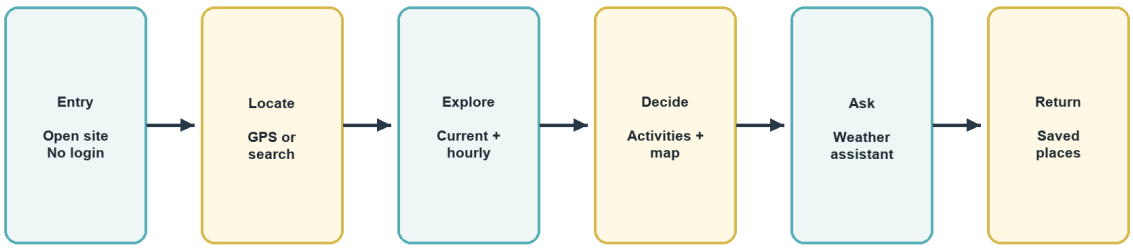


Figure 3: Proposed customer journey map

Stage	Action	Goal	Touchpoint	Pain point	Opportunity
Entry	Open site	Get weather now	Root dashboard	Location may be unclear	Show selected city clearly
Locate	Use GPS/search	Find relevant place	Location/search control	Similar names	Show country/region
Explore	Read cards	Understand now/next hours	Current and hourly cards	Information density	Preserve hierarchy
Decide	Review activities/map	Plan activity	Activity icons/map	Suggestions are informational	Label clearly
Ask	Send question	Interpret data	Chat popup	External provider limits	Show provider/fallback status
Return	Open saved place	Compare quickly	localStorage rows	Device-specific data	Optional account sync later

Figure 4 User Flow Diagram

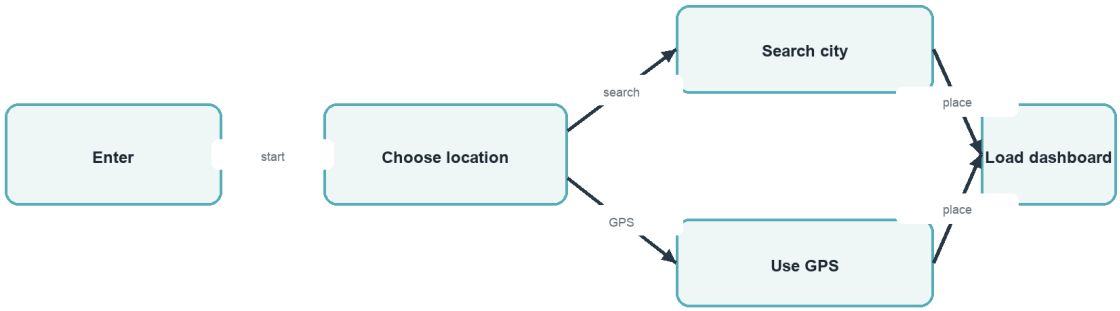


Figure 4: User flow from entry to selected weather dashboard

10 UML and Structural Diagrams

10.1 Activity Diagram

The activity model shows how current weather can inform a simple activity suggestion. It is informational and is not a safety or medical decision engine.

Figure 5 Weather Activity Diagram

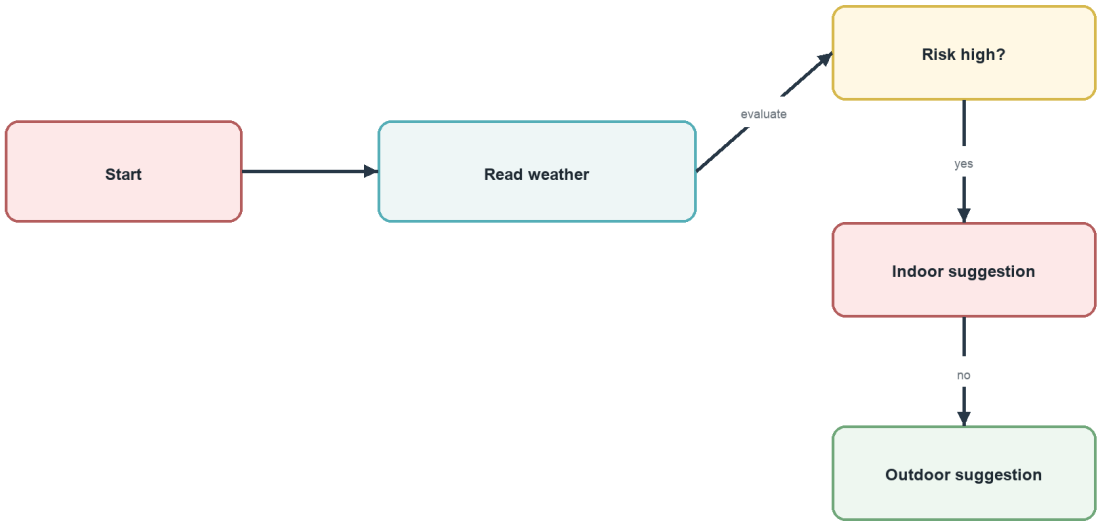


Figure 5: Activity diagram for an activity suggestion

10.2 Sequence Diagram

The sequence diagram shows search and chat interactions across the visitor, React UI, Express API and external providers.

Figure 6 Search and Chat Sequence Diagram

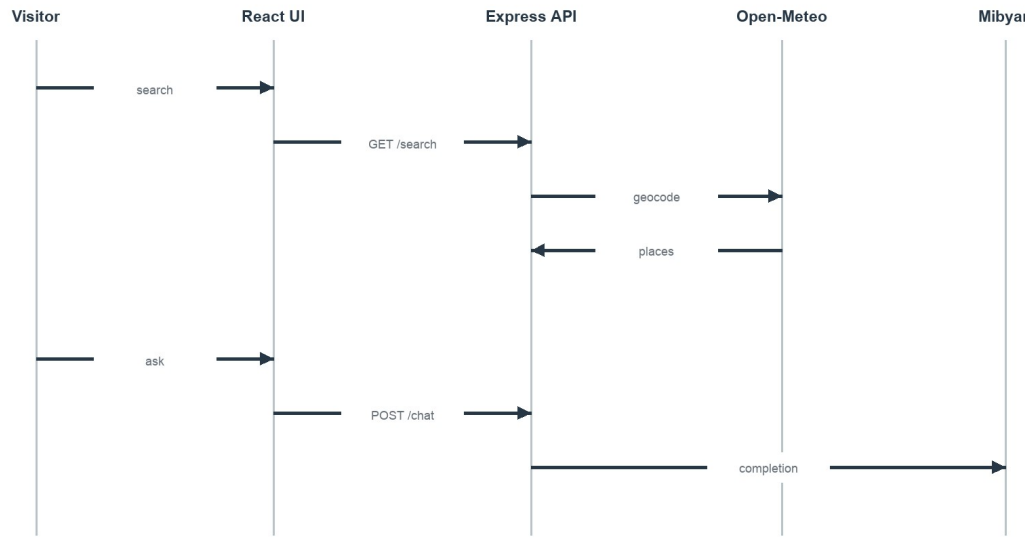


Figure 6: Search and chat sequence diagram

10.3 Component Diagram

Because the project is functional React/TypeScript rather than a class-based object model, a component diagram is more accurate than an invented class diagram.

Figure 7 Component Diagram

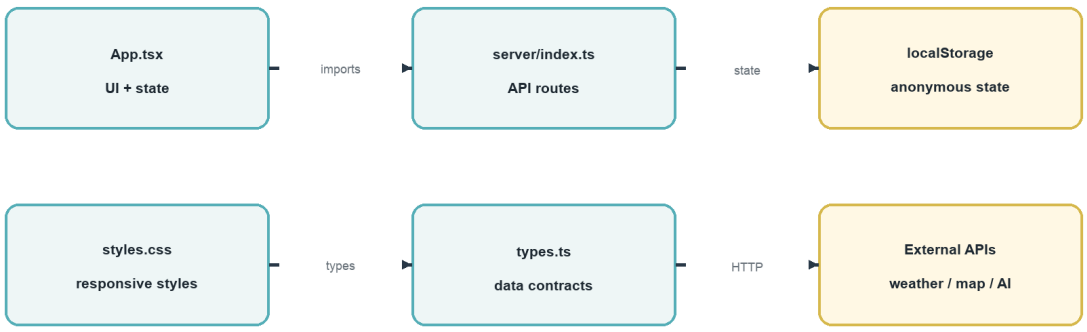


Figure 7: Component diagram of the implemented code structure

11 User Interface and UX Design

The visual language is a light, card-based dashboard inspired by the supplied Atmos Weather reference. It uses a full-width canvas, rounded paper cards, restrained grey surfaces, dark ink typography and teal accents. The top bar combines branding, theme/unit controls, city search and current-location access. The main area prioritises today’s view, a five-card viewport over a horizontally scrollable 24-hour strip, the current-weather card, activities, map and saved places.

At phone width the dashboard becomes a vertical flow. The hourly strip keeps its own horizontal scroll so the document itself does not expand. The chat becomes a bounded popup. Browser inspection at a 539px viewport showed document width equal to viewport width while the hourly strip retained a larger internal scroll width.

Accessibility measures include readable contrast, semantic buttons, Escape-to-close search, aria labels on key chat controls and a design that does not require colour alone to understand weather values. A complete WCAG and screen-reader audit was not executed.

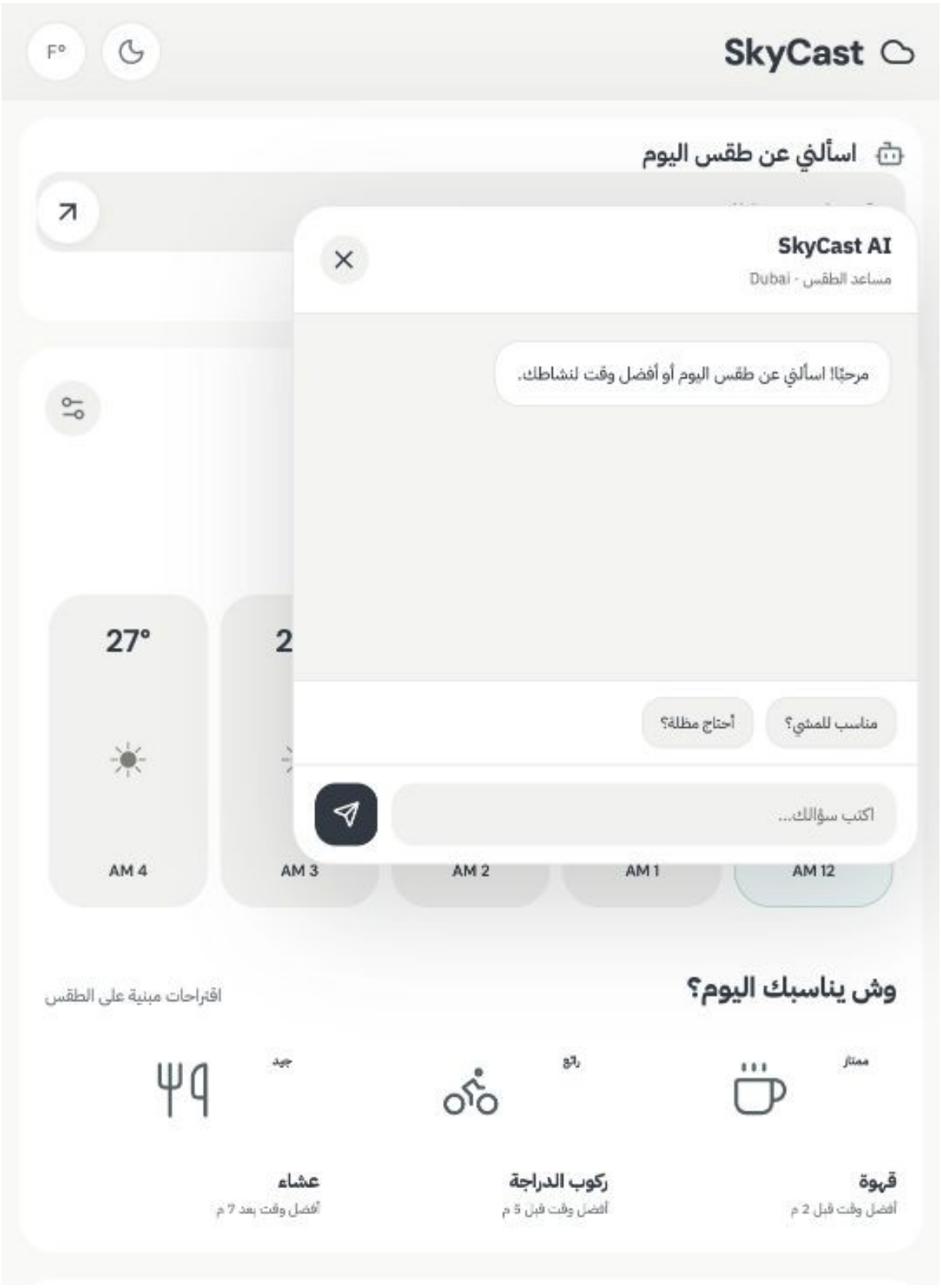


Figure 9: Actual mobile browser screenshot of the dashboard and chat popup

12 Database Design

No database was found. There are no migrations, collections, server-side user records or authentication routes. Saved places, selected place and theme are stored in browser localStorage. This is suitable for anonymous local persistence but is device-specific and cannot synchronise between devices.

12.1 Proposed Future Database

If accounts are later required, a future schema could contain User, SavedPlace, Preference and ChatSession entities with strict ownership checks. This is a proposal only and is not an implemented database.

13 Technology Stack

Technology	Purpose	Where used	Justification
React	Interactive UI	src/App.tsx	Component rendering and hooks
TypeScript	Types and checks	src, server	Catches interface mistakes early
Vite	Dev/build tooling	package scripts	Fast feedback and bundling
Node/Express	Server API	server/index.ts	Protects keys and centralises calls
Open-Meteo	Weather/geocoding	API routes	Forecast payload
Mibyan AI	Chat	/api/chat	OpenAI-compatible endpoint
Leaflet/OSM	Map	WeatherMap	Interactive marker map
Lucide React	Icons	App.tsx	Reusable SVG icons
localStorage	Local persistence	browser effects	No DB needed currently

14 Software Implementation

14.1 Structure

App.tsx contains the main composition, state and interaction handlers. styles.css contains the visual system and responsive overrides. types.ts defines Place and WeatherData. server/index.ts contains Express routes and external-provider calls. package.json provides client, server, combined development and production build scripts.

File	Responsibility
src/App.tsx	Dashboard, state, weather/search/chat interactions, Metric and WeatherMap
src/styles.css	Typography, cards, responsive layout, theme and chat
src/types.ts	Place and WeatherData contracts
server/index.ts	Health, search, weather, reverse-geocode and chat routes
.env/.env.example	Runtime configuration and key placeholders
package.json	Dependencies and scripts

14.2 State and Data

Hooks manage selected place, favourites, weather, search text, unit, theme, chat messages and typing state. Effects load weather after a place change, debounce search, load favourite temperatures, write localStorage and scroll the chat to the latest message.

14.3 API Routes

Route	Method	Purpose	Dependency
/api/health	GET	Health/config response	None
/api/search	GET	Arabic/English place search	Open-Meteo geocoding
/api/weather	GET	Current/hourly/daily forecast	Open-Meteo
/api/reverse-geocode	GET	Coordinate area name	Nominatim
/api/chat	POST	Weather assistant	Mibyan or fallback

14.4 Error Handling and Reuse

The client catches failed fetches and prevents a failed request from crashing the page. The server validates coordinates and uses a bounded chat fallback. Metric removes repeated metric markup, WeatherMap isolates map lifecycle, environment variables keep provider configuration out of browser code, and CSS variables reuse theme values. App.tsx is still relatively concentrated and should be decomposed as the feature set grows.

15 Software Construction Fundamentals

15.1 Minimizing Complexity

The system avoids accounts and a database because the current anonymous workflow does not need them. Its API surface is small, although App.tsx is now the main complexity hotspot.

15.2 Anticipating Change

Provider URLs and model names are environment-configurable. The API boundary permits upstream replacement without rewriting every UI card.

15.3 Constructing for Verification

The build performs TypeScript and Vite checks; the health endpoint exposes a simple diagnostic; browser checks cover Arabic search, phone layout, internal hourly scrolling and chat behaviour.

15.4 Reuse

React, Express, Leaflet and Lucide are reused libraries. Metric, WeatherMap, CSS variables and typed data shapes reduce repetition.

15.5 Standards in Construction

The project uses TypeScript, npm scripts, a src/server separation and environment-based secrets. Automated unit tests, linting and a formal schema are not yet present.

16 Testing Strategy

Executed checks included the production build, health/weather/reverse-geocoding requests, Arabic search, browser interaction, mobile layout measurements and provider-failure diagnosis. No automated unit suite, coverage report, load test, formal accessibility audit or penetration test was found.

ID	Req.	Test	Precondition	Expected	Actual	Status
TC-01	FR-01	Load dashboard	Open root	Weather cards render	Rendered in browser	Passed
TC-02	FR-02	Search Arabic “دبي”	Enter text	Result includes country	Returned and rendered	Passed
TC-03	FR-04	Use GPS	Permission	Area name appears	Permission-dependent	Partial
TC-04	FR-05	Hourly strip	Phone width	Only strip scrolls	Body width equals viewport	Passed
TC-05	FR-06	Save place	Result exists	Row remains after refresh	Implemented	Passed
TC-06	FR-08	Map marker	Tiles available	Marker follows selection	Implemented	Partial
TC-07	FR-09	Mibyan response	Valid project key	Model completion	Current key rejected	Failed prerequisite
TC-08	FR-10	Provider failure	Invalid/limited key	Fallback appears	Observed	Passed
TC-09	FR-09	Chat scroll	Several messages	Last message visible	Auto-scroll verified	Passed
TC-10	NFR	Production build	Dependencies	Build exits cleanly	npm run build passed	Passed

16.1 Coverage and Bug Tracking

No coverage percentage is claimed. Future coverage should target route validation, search deduplication, localStorage, loading/error states, provider failures and map cleanup. A future bug record should contain ID, reproduction, expected/actual result, severity, fix and regression test.

17 Software Quality Evaluation

Dimension	Assessment	Classification
Correctness	Core weather/search flows are implemented; AI depends on key	Partially verified
Readability	Names are clear; App.tsx is dense	Partially verified
Maintainability	Server boundary and env config are useful	Partially verified
Reliability	Chat fallback prevents total failure	Verified for fallback
Usability	No login, disambiguation and mobile containment	Verified for tested view
Performance	No benchmark	Not tested
Security	Server-side key handling; no formal audit	Partially verified
Compatibility	Phone viewport checked; device matrix incomplete	Partially verified

18 Security and Privacy

Mibyan credentials are read by Express from environment variables and are not sent to the React browser. Coordinate and query inputs are validated before provider calls. CORS is permissive for local development and should be restricted for deployment. Location is used to retrieve weather and reverse geocoding; a future analytics or account layer would require clear privacy choices. No authentication or authorization exists because no accounts exist.

19 Challenges and Limitations

- No database, accounts or cross-device synchronisation.
- Weather values depend on upstream provider location, timing and model.
- The map is a base map with marker; weather raster layers and storms are not implemented.
- Mibyan adapter exists, but the current project key returned HTTP 401 during verification.
- Fallback chat is bounded and is not equivalent to model reasoning or web search.
- App.tsx needs feature extraction as scope grows.
- No automated tests, coverage, load, accessibility or security audit was found.

20 Future Improvements

Improvement	Problem	Benefit	Complexity	Priority
Validate/rotate Mibyan key	401 prevents live completions	Real AI answers	Low	High
Extract service modules	Dense App.tsx	Easier tests/providers	Medium	High
Automated tests	Manual verification only	Regression evidence	Medium	High
Weather map layers	Base marker map	Storm/rain context	High	Medium
Comparison view	Limited saved-row comparison	Better planning	Medium	Medium
Account sync	Device-only storage	Cross-device favourites	High	Low
Rate limiting/schema validation	Public routes	Safer deployment	Medium	High
Accessibility audit	Partial checks	Better inclusive access	Medium	Medium

21 Conclusion

Al-Tarqi SkyCast demonstrates a complete small-scale web construction project with a clear visitor workflow: enter the dashboard, identify a place, read current and hourly weather, save places, inspect a map and ask for an interpretation. The solution is grounded in React/TypeScript, Node/Express, Open-Meteo, Leaflet/OpenStreetMap, localStorage and an OpenAI-compatible Mibyan adapter.

The project demonstrates practical Software Construction through API boundaries, environment configuration, reusable UI functions, responsive containment, error handling and iterative verification. The report also preserves an important engineering distinction: the Mibyan integration is implemented according to its documented contract, but the current key failed authentication, so live model quality is not claimed. The fallback provides continuity but remains limited.

22 References

Mibyan AI. (n.d.). API overview. https://docs.mibyanai.com/api-reference/introduction

Mibyan AI. (n.d.). API quickstart. https://docs.mibyanai.com/api-reference/quickstart

Open-Meteo. (n.d.). Weather API documentation. https://open-meteo.com/en/docs

Open-Meteo. (n.d.). Geocoding API documentation. https://open-meteo.com/en/docs/geocoding-api

Leaflet. (n.d.). Leaflet reference. https://leafletjs.com/reference.html

OpenStreetMap Foundation. (n.d.). Copyright and attribution. https://www.openstreetmap.org/copyright

React. (n.d.). React documentation. https://react.dev/

TypeScript. (n.d.). TypeScript handbook. https://www.typescriptlang.org/docs/handbook/intro.html

Vite. (n.d.). Vite guide. https://vite.dev/guide/

Express. (n.d.). Express documentation. https://expressjs.com/

MDN Web Docs. (n.d.). Window localStorage. https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage

MDN Web Docs. (n.d.). Geolocation API. https://developer.mozilla.org/en-US/docs/Web/API/Geolocation_API

Appendix A Technical Details

Item	Detail
Weather	GET /api/weather?latitude=...&longitude=...
Search	GET /api/search?q=...
Chat	POST /api/chat with question, weather and location JSON
Mibyan config	MIBYAN_API_KEY, MIBYAN_BASE_URL, MIBYAN_MODEL
Storage	skycast:place, skycast:favorites, skycast:theme
Database	None found in current implementation

Appendix B Evidence Notes

This report was generated from the project source files, package configuration, executed build/API checks, browser inspection at a phone-sized viewport and the captured mobile screenshot. Student and course metadata remain editable placeholders because they were not supplied.

Appendix C Diagram Index

01_system_architecture.png

02_use_case.png

03_customer_journey.png

04_user_flow.png

05_activity.png

06_sequence.png

07_component.png

08_development_cycle.png